

Project Proposal

A Localized Academic Certificate Verification System using Cryptographic Hashing and Relational Databases

Objective

Problem:Academic degree forgery and manual background checks create immense operational friction, requiring employers to communicate directly with institutions to confirm credential authenticity.

Solution:Automate validation and eliminate tampering by migrating trust to an immutable local ledger system. Universities issue tamper-proof digital certificates that recruiters can instantly verify through a zero-trust model.

Abstract

This project delivers a streamlined, locally hosted web-based document verification ecosystem adapted from the architectural goals outlined in project-proposal.2(1) (2).pdf. Instead of relying on complex decentralized cloud infrastructure, this system utilizes a highly optimized local stack built with vanilla web technologies. Universities generate authentic digital credentials, which are natively hashed via SHA-256 and registered within a local database layer for absolute permanence. Graduating students receive unalterable files to share with employers. Recruiters upload documents to a local public portal for real-time cryptographic matching, reducing verification from weeks to seconds.

Technologies & Tools (The Simplified Stack)

Frontend UI:Vanilla HTML5 + CSS3 (Zero frameworks or build tools for lightweight, native browser rendering).

Backend Logic:Vanilla PHP (Handles file processing, native cryptographic operations, and server-side routing).

Cryptographic Hashing Engine:PHP Native Hashing Core (hash_file utilizing SHA-256 algorithmic structures).

Database & Ledger:MySQL (Local relational database acting as the structured anchor system for certificate records).

Local Server Environment:XAMPP / Apache (For local orchestration of the PHP server and MySQL instance).

Key Features

Institutional Issuance Dashboard:A secure local portal for university registrars to input student metadata, upload degree criteria, and generate tamper-proof credentials.

Cryptographic Anchoring Engine:Server-side SHA-256 fingerprint generation mapped directly with student records inside the MySQL database.

Instant Wallet-Free Verification:Public verification landing page featuring a drag-and-drop file interface allowing immediate validation without third-party integrations.

Tamper Alert System:High-visibility indicator that flags invalid files if any underlying PDF content or metadata is modified post-issuance.

Target Users

University Registrars & Administrators:To safely issue and manage authentic digital credentials locally.

Corporate Recruiters & HR Specialists:To screen candidates and execute background checks instantly.

Graduating Students & Job Applicants:To securely store and distribute un-falsifiable proof of academic achievements.

Timeline & Milestones

Phase	Timeline	Deliverable / Core Activity
1: UI & Environment Setup	Week 1	Configure XAMPP local server, build vanilla HTML/CSS frontend templates.
2: DB Architecture	Week 2	Design and deploy the local MySQL schema; establish secure PHP Data Objects (PDO) connections.
3: Cryptographic Logic	Week 3	Implement native PHP SHA-256 file hashing workflows and upload processing.
4: Portal Integration	Week 4	Connect the frontend forms to the backend verification and issuance scripts.
5: Data Flow Testing	Week 5	Conduct full cross-architectural data flow testing, edge-case debugging, and security audits.

Team Roles

Noshin Nawal (Frontend Architecture):Responsible for vanilla HTML5 layouts, CSS3 styling, UI state forms, and ensuring local cross-browser validation interfaces work seamlessly.

Mohammad Safwan Uddin Saad (Backend & Cryptography):Responsible for the core PHP server-side logic, file upload validation constraints, and implementing the native SHA-256 cryptographic verification engine.

Abdullah Al Muhim (Database & Integration):Responsible for designing the MySQL relational schemas, managing local indexing via phpMyAdmin, and wiring database connection logic to the backend.

Challenges & Risks

Data Privacy: Storing raw PDFs locally can create unnecessary storage bloat and administrative vulnerabilities. Mitigation: The system discards the raw PDF immediately after the server generates the unique SHA-256 hash, keeping only the anonymous alphanumeric fingerprint stored in the MySQL database.

Input Vulnerabilities: Web forms processing file uploads are vulnerable to SQL injection or malicious file execution. Mitigation: Enforce strict server-side file type restrictions (PDF only) and utilize parameterized PDO queries for all MySQL interactions.

Expected Outcomes

This revised local architecture successfully simplifies the advanced mechanics of project-proposal.2(1) (2).pdf into a highly maintainable, academic-friendly environment. It replaces high-barrier frameworks with standard web fundamentals while still delivering a fully functional zero-trust cryptographic validation framework. Operational costs and setup complexity are reduced to zero, making it an ideal demonstration of data integrity principles.